In this case, the AI may suggest modifying an existing permit policy such as `policy-customer-view.cedar` by adding an additional condition to the `when` clause that explicitly excludes this principal and document. This typically appears as an extra conjunct that narrows the circumstances under which the permit applies.

At this point, you might further refine the discussion by asking:

> *"What if I wanted to do this by adding an unless clause instead of putting a conjunction in the when clause?"*

The AI may then refactor the proposed change to use an `unless` clause that cleanly expresses the exception. This version often reads more clearly, especially when the intent is to describe a general rule with a specific carve-out.

Each of these alternatives may be syntactically valid and semantically equivalent, but they are not interchangeable from a design perspective. Choosing between a new `forbid` policy, a narrower `when` clause, or a more readable `unless` clause is ultimately a human judgment about clarity, intent, and long-term maintainability. These are decisions about how authority should be expressed, not questions a language model can answer on its own.

This is the core of a human-in-the-loop workflow. The AI can generate options, surface trade-offs, and refactor logic, but it does not decide which policy best reflects organizational intent. The final responsibility for authorization behavior remains with the human reviewer, who must understand and accept the consequences of each change before it is applied.

B.6 Steps for modifying a policy set to achieve a specific outcome

Policy changes should begin with an intended outcome, not with code. In this section, you will follow a structured approach for modifying a policy set to achieve a narrowly defined authorization result, using AI assistance to explore options while keeping decision-making under human control.